



**UNIVERSIDAD  
SERGIO ARBOLEDA**

Terra - Lenguaje de dominio específico para datos  
ambientales y de sostenibilidad

## **Lenguajes de programación**

**Yslen Natalia Moreno Gutierrez  
David Andrés Castellanos Angulo  
Santiago Patiño Sarmiento**

*Universidad Sergio Arboleda  
Escuela de ciencias exactas e ingeniería  
Bogotá D.C., 09 de septiembre de 2026*

# Índice

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                | <b>4</b>  |
| 1.1. Qué cubre esta entrega . . . . .                 | 4         |
| 1.2. Usuarios, entradas y salidas . . . . .           | 5         |
| <b>2. Filosofía de diseño</b>                         | <b>5</b>  |
| <b>3. Decisiones generales de diseño</b>              | <b>5</b>  |
| <b>4. Tipos de datos</b>                              | <b>6</b>  |
| 4.1. Qué significa tipado dinámico y fuerte . . . . . | 7         |
| <b>5. Palabras reservadas</b>                         | <b>7</b>  |
| <b>6. Operadores</b>                                  | <b>9</b>  |
| 6.1. Tabla de operadores . . . . .                    | 9         |
| 6.2. Precedencia . . . . .                            | 9         |
| <b>7. Estructuras de control</b>                      | <b>10</b> |
| 7.1. Condicional . . . . .                            | 10        |
| 7.2. Ciclo mientras . . . . .                         | 11        |
| 7.3. Ciclo para . . . . .                             | 11        |
| <b>8. Instrucciones del dominio</b>                   | <b>11</b> |
| 8.1. Carga . . . . .                                  | 12        |
| 8.2. Tuberías . . . . .                               | 12        |
| 8.3. Escritura y visualización . . . . .              | 12        |
| <b>9. Manejo de errores</b>                           | <b>13</b> |
| <b>10.Arquitectura del intérprete</b>                 | <b>13</b> |
| 10.1. El recorrido de un programa . . . . .           | 13        |
| 10.2. Por qué el patrón Visitor . . . . .             | 14        |
| 10.3. Módulos . . . . .                               | 14        |
| <b>11.Uso del intérprete</b>                          | <b>15</b> |

|                                              |           |
|----------------------------------------------|-----------|
| <b>12.Programas de ejemplo</b>               | <b>15</b> |
| 12.1. Funciones incorporadas . . . . .       | 16        |
| <b>13.Pruebas</b>                            | <b>16</b> |
| <b>14.Limitaciones actuales</b>              | <b>17</b> |
| <b>15.Trabajo de las siguientes entregas</b> | <b>18</b> |

# 1. Introducción

Un **lenguaje de dominio específico** (DSL) es un lenguaje diseñado para resolver los problemas de un campo delimitado, en vez de servir para cualquier tarea de programación. Python o Java son lenguajes de propósito general: sirven para todo, pero obligan a escribir mucho código para cualquier cosa. Un DSL renuncia a esa amplitud a cambio de que las tareas de su dominio se expresen en una o dos líneas.

**Terra** es el DSL que presenta este documento. Su dominio son los datos ambientales y de sostenibilidad: series de emisiones de CO<sub>2</sub>, participación de energías renovables, indicadores por país y por año. El usuario escribe qué transformación quiere hacer sobre los datos y el intérprete resuelve cómo ejecutarla.

## 1.1. Qué cubre esta entrega

Esta primera entrega corresponde a la fase de *especificación y front-end*: el diseño del lenguaje y la construcción de sus analizadores léxico y sintáctico. Concretamente, se implementó y está funcionando:

- la gramática completa en ANTLR v4, que compila sin advertencias;
- el lexer y el parser generados para Python, con el patrón Visitor;
- el sistema de tipos, los operadores y las estructuras de control, ya evaluándose;
- el reconocimiento sintáctico de las instrucciones de datos y de visualización;
- un manejador de errores propio que reporta en español, con línea y columna;
- una interfaz de línea de comandos con modo archivo, modo interactivo, validador de sintaxis, volcado de tokens y volcado del árbol de análisis;
- una suite de 100 pruebas automatizadas, todas aprobadas.

Las instrucciones de datos (cargar un CSV, filtrar, agrupar) se reconocen pero todavía no se ejecutan, y las gráficas no se producen: eso corresponde a las entregas segunda y tercera.

## 1.2. Usuarios, entradas y salidas

| Aspecto          | Definición                                                                                                                                                                                 |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Usuario objetivo | Analistas ambientales, estudiantes e investigadores que trabajan con datos de emisiones y energía, y que necesitan reproducir un análisis sin escribir un programa completo.               |
| Entradas         | Archivos de texto con extensión <code>.terra</code> que contienen el programa, y archivos CSV con los datos.                                                                               |
| Salidas          | Valores impresos en consola, archivos CSV con los resultados y, desde la tercera entrega, gráficas en PNG.                                                                                 |
| Restricciones    | Sin librerías externas de análisis: toda la lógica de datos, estadística y visualización se escribe desde cero. Las únicas dependencias son el runtime de ANTLR y pytest para las pruebas. |

## 2. Filosofía de diseño

Cuatro ideas guiaron cada decisión del lenguaje.

**Primera: el DSL no debe convertirse en una copia de Python.** Su valor está en ofrecer instrucciones compactas y cercanas al vocabulario de la ciencia de datos. Por eso Terra tiene `agrupar por` y `resumir` como parte de la gramática, no como funciones de una librería.

**Segunda: cada operación produce un dato nuevo.** Una tubería toma una tabla, la transforma y devuelve otra distinta, sin modificar la anterior. Esto hace que un análisis se pueda leer de arriba abajo y que sea reproducible: los mismos datos y el mismo programa dan siempre el mismo resultado.

**Tercera: los errores se detectan temprano y se explican.** El tipado es fuerte y sin conversiones implícitas, y todos los mensajes van en español con la línea y la columna del problema.

**Cuarta: todo se construye desde cero.** No se usan pandas, NumPy ni Matplotlib. La media, la mediana, la desviación estándar y la raíz cuadrada están escritas a mano en el evaluador.

## 3. Decisiones generales de diseño

Esta tabla resume las decisiones que definen el carácter del lenguaje. Cada una se explica en detalle en la sección correspondiente.

| Característica      | Decisión                                                                    | Justificación                                                                                                                                 |
|---------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Paradigma           | Declarativo con tuberías ( <code> &gt;</code> ) y ciclos controlados        | El análisis se lee como una secuencia de transformaciones; los ciclos se conservan para cálculos que la tubería no cubre                      |
| Palabras reservadas | En español                                                                  | El dominio (informes e indicadores ambientales) se documenta en español; el enunciado propone verbos en español para las operaciones de datos |
| Operadores lógicos  | Simbólicos: <code>&amp;&amp;</code> , <code>  </code> , <code>!</code>      | Evita reservar las letras <code>y</code> y <code>o</code> , que son nombres de variable frecuentes en contextos matemáticos                   |
| Tipado              | Dinámico y fuerte                                                           | No hace falta declarar el tipo, pero tampoco hay conversiones silenciosas que oculten errores                                                 |
| Inmutabilidad       | <code>fixo</code> inmutable, <code>var</code> mutable; ninguna se redeclara | Estilo funcional por defecto, mutable solo cuando se necesita                                                                                 |
| Cierre de bloques   | Palabra clave <code>fin</code>                                              | Consistente en <code>si</code> , <code>mientras</code> y <code>para</code> ; no depende de la indentación                                     |
| Extensión           | <code>.terra</code> obligatoria                                             | Identifica los archivos que el intérprete acepta                                                                                              |
| Comentarios         | <code>#</code> de línea, <code>/- -/</code> de bloque                       | <code>#</code> es el estándar de facto en scripts de datos                                                                                    |
| Dependencias        | Solo ANTLR y pytest                                                         | Toda la lógica del dominio se escribe desde cero                                                                                              |

Los operadores lógicos los tomamos como símbolos. El ejemplo del enunciado usa la palabra `y` como conjunción lógica: `filtrar donde unidades >0 y precio >0`. Se descartó esa opción.

En ANTLR, cualquier palabra reservada deja de estar disponible como identificador, porque el lexer la reconoce primero. Si `y` fuera palabra reservada, un programa no podría declarar `fixo y = 5`, algo natural en un lenguaje con vocación matemática, donde `x` e `y` son los nombres más comunes para un par de variables. Lo mismo ocurre con `o`.

Los símbolos `&&`, `||` y `!` no compiten con ningún identificador posible y son familiares para cualquiera que haya visto C, Java o JavaScript. El mismo criterio explica que los ejes de una gráfica se escriban `eje_x` y `eje_y` en vez de `x` y `y`.

## 4. Tipos de datos

Terra tiene seis tipos. El tipo no se declara: se deduce del valor que se asigna. Pero una vez deducido, es estricto.

## 4.1 Qué significa tipado dinámico y fuerte

| Tipo     | Descripción                   | Ejemplo                |
|----------|-------------------------------|------------------------|
| numero   | Entero                        | fijo anio = 2023       |
| decimal  | Punto flotante                | fijo co2 = 88.1        |
| texto    | Cadena entre comillas dobles  | fijo pais = "Colombia" |
| booleano | verdadero o falso             | fijo cumple = falso    |
| nulo     | Ausencia de valor             | fijo sin_dato = nulo   |
| lista    | Secuencia ordenada, indexable | fijo a = [2019, 2020]  |

### 4.1. Qué significa tipado dinámico y fuerte

Son dos propiedades distintas que suelen confundirse.

**Dinámico** quiere decir que el tipo se conoce al ejecutar, no al escribir. No hay que anotar `numero x = 5`: basta `fijo x = 5` y el intérprete deduce que es un número.

**Fuerte** quiere decir que el lenguaje no convierte tipos por su cuenta. En JavaScript, `"5"+1` devuelve `"51"` porque el intérprete convierte el número en texto sin avisar. En Terra eso es un error:

```
fijo x = "bosque" + 1
# Error: no se puede sumar texto con numero
```

En datos ambientales esa severidad es deseable: si una columna de emisiones se leyó como texto por un error de formato, es mejor que el programa se detenga a que produzca un promedio sin sentido.

Para los booleanos hubo que hacer una validación explícita para los booleanos. En Python, el tipo `bool` es una subclase de `int`: `verdadero + 1` devolvería `2` sin protestar. Como Terra se implementa sobre Python, esa herencia se filtraría al lenguaje. La función `es_numero()` del evaluador descarta primero los booleanos y solo después comprueba si el valor es entero o decimal.

## 5. Palabras reservadas

Terra reserva 43 palabras. Ninguna puede usarse como nombre de variable.

| Palabra                     | Categoría          | Función                                             |
|-----------------------------|--------------------|-----------------------------------------------------|
| <b>fixo</b>                 | Declaración        | Declara una variable inmutable                      |
| <b>var</b>                  | Declaración        | Declara una variable mutable                        |
| <b>si</b>                   | Control            | Abre un condicional                                 |
| <b>sino</b>                 | Control            | Alternativa; con <b>si</b> forma el caso intermedio |
| <b>fin</b>                  | Control            | Cierra cualquier bloque                             |
| <b>mientras</b>             | Control            | Ciclo con condición                                 |
| <b>para</b>                 | Control            | Ciclo de recorrido                                  |
| <b>en</b>                   | Control / operador | Recorre una colección; también es pertenencia       |
| <b>mostrar</b>              | Salida             | Imprime uno o varios valores                        |
| <b>cargar</b>               | Datos              | Lee un archivo CSV                                  |
| <b>guardar</b>              | Datos              | Escribe un resultado en disco                       |
| <b>como</b>                 | Datos              | Introduce el destino o el tipo destino              |
| <b>con, separador</b>       | Datos              | Configuran la lectura del CSV                       |
| <b>seleccionar</b>          | Tubería            | Escoge columnas                                     |
| <b>filtrar, donde</b>       | Tubería            | Conserva las filas que cumplen una condición        |
| <b>crear</b>                | Tubería            | Agrega una columna calculada                        |
| <b>renombrar</b>            | Tubería            | Cambia el nombre de una columna                     |
| <b>ordenar, por</b>         | Tubería            | Ordena por una columna                              |
| <b>asc, desc</b>            | Tubería            | Sentido del orden                                   |
| <b>eliminar, duplicados</b> | Tubería            | Descarta filas repetidas                            |
| <b>nulos, rellenar</b>      | Tubería            | Tratan los valores faltantes                        |
| <b>convertir</b>            | Tubería            | Cambia el tipo de una columna                       |
| <b>agrupar</b>              | Tubería            | Agrupar por una o varias columnas                   |
| <b>resumir</b>              | Tubería            | Calcula agregaciones sobre los grupos               |
| <b>limitar</b>              | Tubería            | Conserva las primeras filas                         |
| <b>graficar</b>             | Visualización      | Abre una instrucción de gráfica                     |
| <b>barras, lineas</b>       | Visualización      | Tipos de gráfica disponibles                        |
| <b>histograma</b>           | Visualización      | Tipos de gráfica disponibles                        |
| <b>dispersion, caja</b>     | Visualización      | Tipos de gráfica disponibles                        |
| <b>eje_x, eje_y</b>         | Visualización      | Columnas de cada eje                                |
| <b>titulo, leyenda</b>      | Visualización      | Rótulos de la gráfica                               |
| <b>numero, decimal</b>      | Tipos              | Tipo destino de <b>convertir</b>                    |
| <b>texto, booleano</b>      | Tipos              | Tipo destino de <b>convertir</b>                    |
| <b>verdadero, falso</b>     | Literales          | Valores booleanos                                   |
| <b>nulo</b>                 | Literal            | Ausencia de valor                                   |

En la gramática, las palabras reservadas se declaran *antes* que la regla IDENTIFICADOR. No es un capricho de estilo.

El lexer de ANTLR sigue dos criterios: primero prefiere la coincidencia más larga y, si hay empate, la regla declarada primero. La palabra **fixo** encaja tanto con el token FIJO como con IDENTIFICADOR, y ambas coincidencias miden cuatro caracteres. El



empate lo rompe el orden: como **FIJO** está declarada antes, gana.

Si el orden se invirtiera, todas las palabras reservadas se leerían como nombres de variable y la gramática dejaría de funcionar.

El mismo criterio aplica a los operadores de dos caracteres. `>=` se declara antes que `>`, aunque en este caso la regla de la coincidencia más larga ya lo resolvería sola. Declararlos primero deja la intención explícita.

## 6. Operadores

### 6.1. Tabla de operadores

| Operador                           | Descripción          | Tipos válidos                                            |
|------------------------------------|----------------------|----------------------------------------------------------|
| <code>+</code>                     | Suma o concatenación | numero+numero, decimal+decimal, texto+texto, lista+lista |
| <code>-</code>                     | Resta                | Solo numéricos                                           |
| <code>*</code>                     | Multiplicación       | Solo numéricos                                           |
| <code>/</code>                     | División             | Solo numéricos; divisor distinto de cero                 |
| <code>%</code>                     | Módulo               | Solo numéricos; divisor distinto de cero                 |
| <code>^</code>                     | Potencia             | Solo numéricos; asocia a la derecha                      |
| <code>&gt; &lt; &gt;= &lt;=</code> | Comparación          | Solo numéricos                                           |
| <code>== !=</code>                 | Igualdad             | Mismo tipo; numero y decimal son comparables             |
| <code>&amp;&amp;</code>            | Conjunción           | Solo booleanos                                           |
| <code>  </code>                    | Disyunción           | Solo booleanos                                           |
| <code>!</code>                     | Negación             | Solo booleanos                                           |
| <code>en</code>                    | Pertenencia          | Lista o texto a la derecha                               |
| <code>[i]</code>                   | Índice               | Lista o texto; índice entero dentro de rango             |
| <code> &gt;</code>                 | Tubería              | Encadena operaciones sobre datos                         |

### 6.2. Precedencia

La precedencia decide qué operación se resuelve primero cuando hay varias en la misma expresión. En Terra no se implementa con tablas de prioridad, sino con la propia estructura de la gramática: cada nivel de precedencia es una regla que llama al nivel siguiente. Las reglas más profundas se evalúan primero.

| Nivel      | Regla       | Operadores | Asociatividad  |
|------------|-------------|------------|----------------|
| 1 (menor)  | tuberia     | >          | Izquierda      |
| 2          | o_logico    |            | Izquierda      |
| 3          | y_logico    | &&         | Izquierda      |
| 4          | igualdad    | == !=      | Izquierda      |
| 5          | relacional  | > < >= <=  | Izquierda      |
| 6          | pertenencia | en         | No asociativo  |
| 7          | suma        | + -        | Izquierda      |
| 8          | termino     | * / %      | Izquierda      |
| 9          | potencia    | ^          | <b>Derecha</b> |
| 10         | unario      | - !        | Derecha        |
| 11 (mayor) | postfijo    | [i]        | Izquierda      |

Dos consecuencias comprobables:

```
mostrar(2 + 3 * 4)      # 14, no 20: la multiplicacion esta
    en un nivel mas profundo
mostrar(2 ^ 3 ^ 2)      # 512, no 64: la potencia asocia a
    la derecha
```

La asociatividad a la derecha de la potencia se logra escribiendo la regla de forma recursiva por su lado derecho: `potencia : unario ('^ potencia)?`. Como la regla se llama a sí misma después del operador,  $2 \wedge 3 \wedge 2$  se agrupa como  $2 \wedge (3 \wedge 2)$ , que es la convención matemática.

## 7. Estructuras de control

Los tres bloques cierran con `fin` y abren con dos puntos al final de la línea de encabezado.

### 7.1. Condicional

```
si emision > 10.0:
    mostrar("Emission alta")
sino si emision == 0.0:
    mostrar("Sin registro")
sino:
    mostrar("Emission moderada")
fin
```

Se escribe `sino si` en dos palabras, no como un token único tipo `elif`. Esto evita inventar una palabra reservada adicional y se lee de corrido. ANTLR resuelve la

construcción sin ambigüedad porque su analizador mira los tokens siguientes antes de decidir qué alternativa tomar.

La condición debe ser booleana. Un número no es una condición válida:

```
si 1:
    mostrar("esto falla")
fin
# Error: la condicion debe ser booleana, se recibio numero
```

## 7.2. Ciclo mientras

```
var anio = 2019
mientras anio <= 2023:
    mostrar("Procesando anio", anio)
    anio = anio + 1
fin
```

El evaluador lleva un contador interno y aborta el ciclo si supera un millón de iteraciones, con un mensaje que sugiere revisar la condición de salida. Es una red de seguridad contra ciclos infinitos.

## 7.3. Ciclo para

Recorre listas y textos:

```
para valor en [12.4, 8.1, 15.7]:
    mostrar(valor)
fin

para letra en "eco":
    mostrar(letra)
fin
```

La variable del ciclo es local: se crea al entrar y desaparece al salir. Si ya existía una variable con ese nombre, se restaura su valor original al terminar. Así un ciclo no puede alterar por accidente el estado del programa.

# 8. Instrucciones del dominio

Estas instrucciones se reconocen sintácticamente en esta entrega. Su ejecución corresponde a las siguientes: cuando aparecen en un programa, el evaluador lanza un mensaje que lo indica de forma explícita.

## 8.1. Carga

```
fijo datos = cargar "datos/emisiones.csv"  
fijo otros = cargar "datos/otros.csv" con separador ";"
```

## 8.2. Tuberías

El operador `|>` toma el resultado de la izquierda y lo pasa como entrada de la operación de la derecha. Cada paso produce una tabla nueva.

```
fijo limpio = datos  
  |> seleccionar [pais, anio, co2, poblacion]  
  |> filtrar donde co2 > 0.0 && anio >= 2015  
  |> eliminar nulos  
  |> crear co2_per_capita = co2 / poblacion  
  |> ordenar por co2 desc
```

La ventaja frente a anidar funciones es el orden de lectura. La versión equivalente en notación de funciones, `ordenar(crear(filtrar(seleccionar(datos))))`, se lee de dentro hacia afuera; la tubería se lee de arriba abajo, en el orden en que ocurren las cosas.

Las agregaciones se aplican sobre grupos:

```
fijo resumen = limpio  
  |> agrupar por [pais]  
  |> resumir emisiones = suma(co2),  
              promedio = media(co2_per_capita),  
              registros = contar()  
  |> limitar 10
```

## 8.3. Escritura y visualización

```
guardar resumen como "salidas/resumen-paises.csv"
```

```
graficar barras resumen  
  eje_x pais  
  eje_y emisiones  
  titulo "Emisiones totales de CO2 por pais"  
  guardar como "salidas/emisiones-pais.png"
```

Las opciones de la gráfica van una por línea y en cualquier orden. Todas son opcionales.

## 9. Manejo de errores

Terra distingue tres clases de error según la etapa en que aparecen.

| Clase      | Cuándo se detecta               | Ejemplo                                                              |
|------------|---------------------------------|----------------------------------------------------------------------|
| Léxico     | Al convertir el texto en tokens | Un carácter que no pertenece al alfabeto del lenguaje                |
| Sintáctico | Al construir el árbol           | si <code>x &gt; 0</code> sin los dos puntos                          |
| Semántico  | Al evaluar el árbol             | <code>a+1</code> ; reasignar un <code>fijo</code> ; dividir por cero |

ANTLR reporta por defecto en inglés y con un formato genérico. La clase `ManejadorErrores` reemplaza ese comportamiento: se registra en el lexer y en el parser, traduce los mensajes más frecuentes y guarda cada error con su línea y su columna. Si hubo errores, el programa no se ejecuta.

Salida real ante un programa con tres errores deliberados:

```

1 Error sintactico [linea 3:5] sobra el simbolo '2' se
   esperaba IDENTIFICADOR
2 Error sintactico [linea 5:11] falta ':' en '\n'
3 Error sintactico [linea 9:20] simbolo inesperado '\n' se
   esperaba {' ',' '}'
4
5 El programa contiene 3 error(es). No se ejecuta.
```

Los errores semánticos se reportan al evaluar, con la línea de la sentencia responsable:

```

1 Error en evaluacion: [linea 2] 'x' se declaro con 'fijo'
   y no puede reasignarse
2 Error en evaluacion: [linea 1] no se puede sumar texto
   con numero
3 Error en evaluacion: [linea 1] indice 5 fuera de rango
   (longitud 2)
```

## 10. Arquitectura del intérprete

### 10.1. El recorrido de un programa

Un archivo `.terra` atraviesa cinco etapas:

1. **InputStream.** El texto se convierte en un flujo de caracteres.
2. **TerraLexer.** El flujo se agrupa en tokens. Aquí `fijo` deja de ser cuatro letras y pasa a ser el token `FIJO`.

3. **CommonTokenStream.** Los tokens se almacenan en un buffer que el parser puede recorrer y adelantar.
4. **TerraParser.** Los tokens se organizan en un árbol de análisis según las reglas de la gramática. Si no encajan, se reporta un error sintáctico.
5. **VisitanteEvaluador.** El árbol se recorre nodo por nodo y se ejecuta la lógica de cada construcción.

## 10.2. Por qué el patrón Visitor

ANTLR ofrece dos formas de recorrer el árbol: Listener y Visitor. El Listener notifica automáticamente la entrada y la salida de cada nodo, pero no devuelve valores ni controla el orden del recorrido. El Visitor sí: cada método devuelve un resultado y decide explícitamente qué hijos visita.

Esa diferencia es decisiva para un intérprete. Evaluar  $2 + 3 * 4$  exige que el nodo de la suma reciba los valores de sus dos hijos y devuelva el resultado hacia arriba. Y ejecutar un **si** exige visitar solo la rama cuya condición se cumplió, no todas. Un Listener no puede hacer ninguna de las dos cosas.

## 10.3. Módulos

| Archivo                    | Responsabilidad                                               |
|----------------------------|---------------------------------------------------------------|
| gramaticas/Terra.g4        | Gramática léxica y sintáctica. Se edita a mano                |
| gramaticas/TerraLexer.py   | Generado por ANTLR. No se edita                               |
| gramaticas/TerraParser.py  | Generado por ANTLR. No se edita                               |
| gramaticas/TerraVisitor.py | Clase base del Visitor, generada. No se edita                 |
| src/terra.py               | Punto de entrada; coordina el pipeline y la línea de comandos |
| src/visitante_evaluador.py | Evaluación semántica y tabla de símbolos                      |
| src/manejador_errores.py   | Errores léxicos y sintácticos en español                      |
| src/validador.py           | Modo interactivo que solo dictamina si una entrada es válida  |
| tests/test_terra.py        | Suite de 100 pruebas                                          |
| generar.sh                 | Regenera lexer, parser y visitor desde el .g4                 |
| arbol.sh                   | Muestra el árbol de análisis de un programa                   |

El evaluador mantiene un diccionario donde cada nombre apunta a su valor y a si admite reasignación. Con esa estructura mínima se implementan tres reglas:

- una variable no puede usarse antes de declararse;
- ningún nombre puede declararse dos veces, sea **fixo** o **var**;
- las declaradas con **fixo** rechazan cualquier reasignación posterior.

## 11. Uso del intérprete

| Comando                                                  | Efecto                                  |
|----------------------------------------------------------|-----------------------------------------|
| <code>python src/terra.py archivo.terra</code>           | Ejecuta el programa                     |
| <code>python src/terra.py</code>                         | Modo interactivo (REPL)                 |
| <code>python src/terra.py archivo.terra --validar</code> | Solo dictamina si la sintaxis es válida |
| <code>python src/terra.py archivo.terra --arbol</code>   | Imprime el árbol de análisis            |
| <code>python src/terra.py archivo.terra --tokens</code>  | Lista los tokens reconocidos            |
| <code>python src/validador.py</code>                     | Validador interactivo de construcciones |
| <code>./arbol.sh archivo.terra -gui</code>               | Dibuja el árbol en una ventana          |
| <code>pytest tests/ -v</code>                            | Ejecuta la suite de pruebas             |

El modo interactivo acepta expresiones de una línea y conserva las variables entre líneas. Los bloques (**si**, **mientras**, **para**) requieren un archivo, porque cada línea del REPL se analiza por separado y un bloque nunca alcanzaría su **fin**.

## 12. Programas de ejemplo

| Archivo                                       | Qué demuestra                                                        |
|-----------------------------------------------|----------------------------------------------------------------------|
| <code>01_variables.terra</code>               | Los seis tipos, <b>fijo</b> y <b>var</b> , salida con <b>mostrar</b> |
| <code>02_operadores.terra</code>              | Precedencia, asociatividad, pertenencia, índices                     |
| <code>03_control.terra</code>                 | Condicional de tres ramas, <b>mientras</b> , <b>para</b>             |
| <code>04_carga_seleccion.terra</code>         | Carga de CSV y preparación con tuberías                              |
| <code>05_indicadores_ambientales.terra</code> | Caso ambiental: agrupación, agregaciones, gráficas                   |
| <code>06_errores.terra</code>                 | Prueba negativa: tres errores sintácticos deliberados                |

Un fragmento representativo, que calcula la intensidad de carbono de cuatro países y los clasifica:

```
fijo paises = ["Colombia", "Peru", "Chile", "Ecuador"]
fijo emisiones = [88.1, 55.6, 85.2, 39.5]
fijo poblaciones = [52.3, 34.2, 19.6, 18.2]
```

```
var i = 0
var per_capita = 0.0
```

```
mientras i < longitud(paises):
    per_capita = emisiones[i] / poblaciones[i]
    si per_capita > 4.0:
```

## 12.1 Funciones incorporadas

```

        mostrar(paises[i], "– intensidad alta:",
                redondear(per_capita, 2))
    sino si per_capita > 2.0:
        mostrar(paises[i], "– intensidad media:",
                redondear(per_capita, 2))
    sino:
        mostrar(paises[i], "– intensidad baja:",
                redondear(per_capita, 2))
    fin
    i = i + 1
fin

mostrar("Promedio regional:", redondear(media(emisiones),
2))
mostrar("Desviacion:", redondear(desviacion(emisiones), 3))

```

### 12.1. Funciones incorporadas

Todas están escritas desde cero en el evaluador, sin recurrir a `math` ni a `statistics`.

| Función                                   | Argumento            | Devuelve                           |
|-------------------------------------------|----------------------|------------------------------------|
| <code>longitud</code>                     | lista o texto        | Cantidad de elementos              |
| <code>suma</code>                         | lista numérica       | Suma de los valores                |
| <code>media</code>                        | lista numérica       | Promedio aritmético                |
| <code>mediana</code>                      | lista numérica       | Valor central de la lista ordenada |
| <code>minimo, maximo</code>               | lista numérica       | Valor extremo                      |
| <code>desviacion</code>                   | lista numérica       | Desviación estándar muestral       |
| <code>raiz</code>                         | número no negativo   | Raíz cuadrada                      |
| <code>absoluto</code>                     | número               | Valor absoluto                     |
| <code>redondear</code>                    | número [, decimales] | Valor redondeado                   |
| <code>a_numero, a_decimal, a_texto</code> | cualquier valor      | Conversión explícita de tipo       |

Las conversiones se llaman `a_numero` y `a_texto`, y no `numero()` ni `texto()`, porque `numero`, `decimal`, `texto` y `booleano` son palabras reservadas de la instrucción `convertir`. El lexer nunca las devolvería como identificadores, así que no podrían usarse como nombre de función.

## 13. Pruebas

La suite contiene 100 pruebas automatizadas con `pytest`, organizadas en ocho grupos:



| Grupo             | Qué verifica                                                                                                               |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| Léxico y sintaxis | Comentarios ignorados, programas válidos aceptados, programas inválidos rechazados, tuberías y <b>graficar</b> reconocidas |
| Variables         | Declaración, reasignación, inmutabilidad, redeclaración, uso sin declarar                                                  |
| Tipos             | Los seis tipos y las listas vacías                                                                                         |
| Aritmética        | Los seis operadores, precedencia, asociatividad, división y módulo por cero                                                |
| Tipado fuerte     | Ocho combinaciones inválidas que deben fallar                                                                              |
| Control de flujo  | Las tres estructuras, anidamiento, localidad de la variable del <b>para</b>                                                |
| Funciones         | Las trece incorporadas y sus casos límite                                                                                  |
| Caso ambiental    | Cálculo per cápita, clasificación de emisiones, promedios de series anuales                                                |

Las pruebas negativas son casi la mitad del total y son deliberadamente numerosas: un intérprete que acepta programas inválidos es tan defectuoso como uno que rechaza programas correctos.

```

1 $ pytest tests/ -q
2 ..... [
3     50%]
4 .....
5     [100%]
6 100 passed in 0.12s

```

## 14. Limitaciones actuales

- Las instrucciones de datos (**cargar**, tuberías, **guardar**) se reconocen pero no se ejecutan.
- Las gráficas no se producen: la instrucción se valida sintácticamente y nada más.
- No hay funciones definidas por el usuario; solo las incorporadas.
- El modo interactivo no admite bloques de varias líneas.
- No hay ámbitos anidados: la tabla de símbolos es única y global.
- No hay entrada de datos por teclado durante la ejecución de un archivo.

## 15. Trabajo de las siguientes entregas

**Segunda entrega.** La estructura `Tabla`, el lector de CSV escrito desde cero, y la ejecución de las tuberías: selección, filtrado, columnas calculadas, tratamiento de nulos, agrupación y agregaciones. Escritura de resultados en CSV. Validaciones semánticas de columnas inexistentes y tipos incompatibles.

**Tercera entrega.** Generación de al menos cuatro tipos de gráfica exportadas a PNG, sin librerías de visualización. Interfaz de línea de comandos definitiva y caso de estudio completo con datos públicos de emisiones y energía.